Mean and variance (÷ n)

0) Given x = −1, 1, 2, 2, 4 (n = 5)

1) Mean μ = add all x, divide by n eg (−1 + 1 + 2 + 2 + 4)/5 = 8/5 = 1.6

2) Each deviation x − μ, then square it eg −2.6, −0.6, 0.4, 0.4, 2.4; squared 6.76, 0.36, 0.16, 0.16, 5.76

3) σ^2 = add the squares, divide by n eg 6.76 + 0.36 + 0.16 + 0.16 + 5.76 = 13.2; 13.2/5 = 2.64 Keys: [MENU] Statistics [=] [1], each x [=], [OPTN 3]: $\bar{x}$ 1.6, $\sigma^2 x$ 2.64

Sanity: deviations add to 0: −2.6 − 0.6 + 0.4 + 0.4 + 2.4 = 0

Trap: $s^2 x$ (÷ n − 1, gives 3.3) is NOT used. [OPTN 2 2] clears

Gaussian naive Bayes: fit

0) Given 10 points, 5 per class. Class 0 (x1, x2): (−1,4), (1,5), (2,2), (2,7), (4,4)

1) Prior = class count ÷ total. eg P(C0) = 5/10 = 0.5

2) Each feature: μ and σ^2 from THIS class's rows only (recipe above) eg x1: −1, 1, 2, 2, 4: μ = 1.6, σ^2 = 2.64 (above) eg x2: (4 + 5 + 2 + 7 + 4)/5 = 22/5 = 4.4; deviations −0.4, 0.6, −2.4, 2.6, −0.4 eg squares 0.16 + 0.36 + 5.76 + 6.76 + 0.16 = 13.2; 13.2/5 = 2.64

3) Write mean vector, diagonal Σ (naive: off-diagonal 0), prior eg μ0 = (1.6, 4.4), Σ0 = [2.64 0; 0 2.64], P(C0) = 0.5. Class 1 same way: μ1 = (3.4, 2.6)

Full (not naive) Σ off-diagonal: cov = (1/n)Σ(x1 − μ1)(x2 − μ2) eg (−2.6)(−0.4) + ... = 1.04 − 0.36 − 0.96 + 1.04 − 0.96 = −0.2; −0.2/5 = −0.04

Trap: divide by the class size nk, not all n

Gaussian naive Bayes: classify a point

0) Given μ0 = (1.6, 4.4), μ1 = (3.4, 2.6), all σ^2 = 2.64, priors 0.5, x = (2, 4)

1) Front = $1/\sqrt{(2\pi\sigma^2)}$ (same for all here) eg 2π × 2.64 = 16.5876; √16.5876 = 4.0728; 1/4.0728 = 0.2455

2) Exponent = $-(x-\mu)^2/(2\sigma^2)$, $2\sigma^2 = 2 \times 2.64 = 5.28$ eg c0: $(2 - 1.6)^2 = 0.16$ and $(4 - 4.4)^2 = 0.16$; −0.16/5.28 = −0.0303 eg c1: $(2 - 3.4)^2 = 1.96$ and $(4 - 2.6)^2 = 1.96$; −1.96/5.28 = −0.3712

3) p(xj) = front × e^exponent ([SHIFT ln], exponent in brackets) eg c0: 0.2455 × e^(−0.0303) = 0.2455 × 0.9702 = 0.2382 (both features) eg c1: 0.2455 × e^(−0.3712) = 0.2455 × 0.6899 = 0.1694 (both features) Table: class | p(x1) | p(x2) | prior | score

4) score = prior × p(x1) × p(x2) eg c0: 0.5 × 0.2382 × 0.2382 = 0.0284; c1: 0.5 × 0.1694 × 0.1694 = 0.0143

5) P(Ck|x) = score ÷ sum of scores eg 0.0284/(0.0284 + 0.0143) = 0.0284/0.0427 = 0.6651 (unrounded 0.6641)

6) Biggest wins. eg class 0

Sanity: posteriors add to 1

Trap: σ^2 (not σ) under the root; $2\sigma^2$ in the exponent

Bayes posterior (two branches)

0) Given P(D) = 0.01, P(+|D) = 0.9, P(+|¬D) = 0.05; find P(D|+)

1) top = P(+|D) × P(D). eg 0.9 × 0.01 = 0.009

2) other = P(+|¬D) × (1 − P(D)). eg 0.05 × (1 − 0.01) = 0.05 × 0.99 = 0.0495

3) P(+) = top + other. eg 0.009 + 0.0495 = 0.0585

4) P(D|+) = top ÷ P(+). eg 0.009/0.0585 = 0.1538

5) Meaning. eg only 15.4% of positives have D

MAP = argmax P(D|h)P(h); MLE = argmax P(D|h) (same if uniform prior)

Trap: forgetting the ¬D branch in P(+)

Cosine similarity, Euclidean distance

0) Given a = (1, 2), b = (2, 1)

1) cos = a·b ÷ (|a| |b|), $|a| = \sqrt{(\Sigma a^2)}$. eg a·b = 1×2 + 2×1 = 4; |a| = |b| = √(1 + 4) = √5 = 2.2361 eg 4/(2.2361 × 2.2361) = 4/5 = 0.8

2) Euclid d = $\sqrt{\Sigma(ai - bi)^2}$. eg $\sqrt{((1-2)^2 + (2-1)^2)} = \sqrt{(1+1)} = \sqrt{2} = 1.4142$

Discrete naive Bayes + Laplace

0) Given tennis, x = (sunny, cool, high, strong); 14 days: 9 yes, 5 no

1) Prior = class count ÷ total. eg yes 9/14 = 0.6429, no 5/14 = 0.3571

2) Each feature: P(value|class) = count of value in class ÷ class count eg yes: sunny 2/9, cool 3/9, high 3/9, strong 3/9 eg no: sunny 3/5, cool 1/5, high 4/5, strong 3/5

3) score = prior × every P(value|class) eg yes: 9/14 × 2/9 × 3/9 × 3/9 × 3/9 = 0.6429 × 0.0082 = 0.0053 eg no: 5/14 × 3/5 × 1/5 × 4/5 × 3/5 = 0.3571 × 0.0576 = 0.0206

4) Biggest wins; P = score ÷ sum. eg No: 0.0206/(0.0053 + 0.0206) = 0.0206/0.0259 = 0.7954

5) Laplace: (count + 1) ÷ (class count + V), V = no. of values of that feature eg sunny|yes, 3 outlook values: (2 + 1)/(9 + 3) = 3/12 = 0.25 Text: (count(w, C) + 1) ÷ (words in C + vocabulary size)

Trap: one zero count makes the whole product 0: use Laplace

Parameter counts (d features, K classes)

Linear / logistic regression: d + 1 (weights + bias)

Softmax: K(d + 1), only (K − 1)(d + 1) free; K = 2 gives d + 1

Gaussian NB: 2dK (μ and σ^2 per feature per class) + (K − 1) priors

Dense layer: (inputs + 1) × outputs. eg 784-128-10: 100480 + 1290

Conv layer: K(F·F·C_in + 1); pooling has 0

Confusion matrix metrics

0) Given TP = 5, FN = 15 (actual +); FP = 30, TN = 50 (actual −); total 100

1) Precision = TP/(TP + FP). eg 5/(5 + 30) = 5/35 = 0.1429

2) Recall = TPR = TP/(TP + FN). eg 5/(5 + 15) = 5/20 = 0.25

3) FPR = FP/(FP + TN). eg 30/(30 + 50) = 30/80 = 0.375; Specificity = 1 − 0.375 = 0.625

4) Accuracy = (TP + TN)/total. eg (5 + 50)/100 = 55/100 = 0.55

5) F1 = 2TP/(2TP + FP + FN). eg 2×5/(2×5 + 30 + 15) = 10/55 = 0.1818 or 2PR/(P + R): 2 × 0.1429 × 0.25 = 0.07145; 0.07145/0.3929 = 0.1819 (unrounded 0.1818)

Sanity: F1 lies between P and R (0.1429 to 0.25)

Trap: check which axis of the matrix is actual

ROC points, invert fix, AUC

0) Given one threshold TP 5, FN 15, FP 30, TN 50; curve (0,0), (0.2,0.6), (0.5,0.8), (1,1)

1) Each threshold: TPR = TP/(TP + FN), FPR = FP/(FP + TN); point (FPR, TPR) eg TPR = 5/20 = 0.25, FPR = 30/80 = 0.375: point (0.375, 0.25) Table: threshold | TP FN FP TN | TPR | FPR

2) Plot x = FPR, y = TPR; add (0,0), (1,1); draw diagonal y = x

3) Below diagonal (TPR < FPR): worse than random. eg 0.25 < 0.375

4) Fix: invert (use 1 − score): (FPR, TPR) becomes (1 − FPR, 1 − TPR) eg (1 − 0.375, 1 − 0.25) = (0.625, 0.75), now above the diagonal

5) AUC: sort by x; each strip = (x2 − x1)(y1 + y2)/2; add strips eg (0.2 − 0)(0 + 0.6)/2 = 0.2 × 0.6/2 = 0.06 eg (0.5 − 0.2)(0.6 + 0.8)/2 = 0.3 × 1.4/2 = 0.21 eg (1 − 0.5)(0.8 + 1)/2 = 0.5 × 1.8/2 = 0.45; AUC = 0.06 + 0.21 + 0.45 = 0.72

6) 0.5 random, 1 perfect; inverted AUC = 1 − old. eg 1 − 0.72 = 0.28

Trap: x is FPR, y is TPR

Bootstrap probability

0) Given n = 10 points, draw 10 times with replacement

1) P(a point missed in 1 draw) = 1 − 1/n. eg 1 − 1/10 = 0.9

2) P(missed in all n draws) = (1 − 1/n)^n. eg 0.9^10 = 0.3487 Keys: (1 − 1 ÷ 10) [x^■] 10 [=]

3) P(in sample) = 1 − that. eg 1 − 0.3487 = 0.6513 eg n = 100: 0.99^100 = 0.3660; 1 − 0.3660 = 0.6340

Sanity: big n tends to 1 − 1/e = 0.632 in bag, 0.368 OOB

Trap: power is n (draws); answer is 1 − (1 − 1/n)^n

CNN output size and parameters

0) Given 32×32×3 input; conv 6 filters 5×5, P = 0, S = 1; then 2×2 pool, S = 2

1) Conv out O = (N + 2P − F)/S + 1; depth = K (no. of filters) eg (32 + 2×0 − 5)/1 + 1 = 27 + 1 = 28: 28×28×6

2) Pool: same formula with F = 2, S = 2; depth unchanged; 0 params eg (28 − 2)/2 + 1 = 13 + 1 = 14: 14×14×6

3) Conv params = K(F × F × C_in + 1); C_in = input depth, +1 bias eg 6(5 × 5 × 3 + 1) = 6(75 + 1) = 6 × 76 = 456 Table: layer | in | F, S, P, K | out | params

4) Compare 2 configs: both as formulas, (new − old), sign: + up, − down Swap s1 > s2 (each conv + 2×2 pool): pool1 (p − s + 1)/2 UP; pool2 (p − s1 − 2s2 + 3)/4, change (s2 − s1)/4 DOWN conv1 f1($3s^2$ + 1) DOWN; conv2 f2($f1s^2$ + 1) UP; total change f1($s1^2 - s2^2$)(f2 − 3) UP eg p 34, s1 7, s2 3, f1 6, f2 16: old 6(3×49 + 1) + 16(6×9 + 1) = 888 + 880 = 1768 eg new 6(3×9 + 1) + 16(6×49 + 1) = 168 + 4720 = 4888; change 3120 = 6(49 − 9)(16 − 3)

Sanity: O is a whole number

Trap: +1 bias per filter; C_in = input depth

k-means: one iteration

0) Given c1 = (4.6, 8.2), c2 = (9.4, 6.8); points A(2,8), B(5,5), C(6,4), D(4,11), F(6,13), H(8,11)

1) d = $\sqrt{((x - cx)^2 + (y - cy)^2)}$ to EACH centre; join the nearest eg C to c1: $(6 - 4.6)^2 + (4 - 8.2)^2 = 1.96 + 17.64 = 19.6$; √19.6 = 4.427 eg C to c2: $(6 - 9.4)^2 + (4 - 6.8)^2 = 11.56 + 7.84 = 19.4$; √19.4 = 4.405: c2 Table: point | d to c1 | d to c2 | cluster A(2,8) | √(6.76 + 0.04) = 2.608 | √(54.76 + 1.44) = 7.497 | c1 B(5,5) | √(0.16 + 10.24) = 3.225 | √(19.36 + 3.24) = 4.754 | c1 H(8,11) | √(11.56 + 7.84) = 4.405 | √(1.96 + 17.64) = 4.427 | c1

2) New centre = (mean of x, mean of y) of its points eg c1 = {A, B, D, F, H}: ((2 + 5 + 4 + 6 + 8)/5, (8 + 5 + 11 + 13 + 11)/5) = (25/5, 48/5) = (5.0, 9.6) Keys: stats [OPTN 3] $\bar{x}$ (x then y, [OPTN 2 2] clear between)

3) Converged only if no point changed cluster. eg C and H switched: No, iterate

Trap: near ties need 3 dp

Entropy, information gain, Gini

0) Given node [2, 3]; split S [29+, 35−] into [21+, 5−] and [8+, 30−]

1) p = class count ÷ node total. eg 2/5 = 0.4, 3/5 = 0.6

2) H = −p1 log2 p1 − p2 log2 p2 (0 log 0 = 0) Keys: log2 x = [log] x [)] ÷ [ALPHA (−)] (A = log 2 stored) eg log2 0.4 = −1.3219, log2 0.6 = −0.7370 eg 0.4 × 1.3219 + 0.6 × 0.7370 = 0.5288 + 0.4422 = 0.9710 Table: class | count | p | log2 p | −p log2 p; add last column

3) Parent H(S). eg log2(29/64) = −1.1420, log2(35/64) = −0.8707 eg 29/64 × 1.1420 + 35/64 × 0.8707 = 0.5175 + 0.4762 = 0.9937

4) Each branch H(Sv), same way. eg [21,5] = 0.7063; [8,30] = 0.7425

5) H(S|A) = Σ (branch size/total) × H(branch) eg 26/64 × 0.7063 + 38/64 × 0.7425 = 0.2869 + 0.4409 = 0.7278

6) IG = H(S) − H(S|A); split on biggest IG. eg 0.9937 − 0.7278 = 0.2659

Gini = 1 − Σp². eg [9,5]: $1 - (9/14)^2 - (5/14)^2 = 1 - 81/196 - 25/196 = 90/196 = 0.4592$

Gini gain = Gini(S) − Σ(size/total)Gini(Sv) eg 0.4956 − (26/64 × 0.3107 + 38/64 × 0.3324) = 0.4956 − (0.1262 + 0.1974) = 0.1720

Gain ratio = IG ÷ SplitInfo, SplitInfo = −Σ(size/total) log2(size/total)

Sanity: 0 ≤ H ≤ 1 (2 classes); IG ≥ 0

Trap: weight branches by size before adding

Softmax

0) Given z = (1, 2, 3)

1) e^z each [SHIFT ln]. eg $e^1 = 2.7183$, $e^2 = 7.3891$, $e^3 = 20.0855$

2) Add. eg 2.7183 + 7.3891 + 20.0855 = 30.1929

3) p = e^z ÷ sum. eg 2.7183/30.1929 = 0.0900, 7.3891/30.1929 = 0.2447, 20.0855/30.1929 = 0.6652

Sanity: 0.0900 + 0.2447 + 0.6652 = 0.9999 (1 up to rounding). Trap: divide e^z, not z

Perceptron update (labels ±1)

0) Given w = (0, 0, 0), x = (1, 1) plus x0 = 1: (1, 1, 1); y = +1; η = 0.5

1) ŷ = +1 if w·x > 0 else −1. eg 0×1 + 0×1 + 0×1 = 0, not > 0: ŷ = −1, wrong

2) If wrong: new w = w + η(y − ŷ)x eg (0,0,0) + 0.5 × (1 − (−1)) × (1,1,1) = 0.5 × 2 × (1,1,1) = (1, 1, 1)

Sanity: now w·x = 3 > 0: ŷ = +1 (right: no change)

Linear regression (1 feature)

0) Given x = 1, 2, 3, 4; y = 2, 3, 5, 6 (n = 4)

1) Means. eg $\bar{x} = 10/4 = 2.5$, $\bar{y} = 16/4 = 4$ Table: x | y | x − x̄ | y − ȳ | product | $(x - \bar{x})^2$

2) Deviations. eg x − x̄: −1.5, −0.5, 0.5, 1.5; y − ȳ: −2, −1, 1, 2

3) Slope $w1 = \Sigma(x - \bar{x})(y - \bar{y}) \div \Sigma(x - \bar{x})^2$ eg (3 + 0.5 + 0.5 + 3)/(2.25 + 0.25 + 0.25 + 2.25) = 7/5 = 1.4

4) Intercept $w0 = \bar{y} - w1\bar{x}$. eg 4 − 1.4 × 2.5 = 4 − 3.5 = 0.5

5) $\hat{y} = w0 + w1x$; $SSE = \Sigma(y - \hat{y})^2$ eg $\hat{y}$ = 1.9, 3.3, 4.7, 6.1; y − ŷ = 0.1, −0.3, 0.3, −0.1; SSE = 0.01 + 0.09 + 0.09 + 0.01 = 0.2

6) $SST = \Sigma(y - \bar{y})^2$. eg 4 + 1 + 1 + 4 = 10

7) $R^2 = 1 - SSE/SST$; MSE = SSE/n. eg 1 − 0.2/10 = 0.98; 0.2/4 = 0.05

GD: $\partial w0 = (2/n)\Sigma(\hat{y} - y)$; $\partial w1 = (2/n)\Sigma x(\hat{y} - y)$; new w = w − α × ∂ eg w = (0, 0), α = 0.1: ŷ = 0, so ŷ − y = −2, −3, −5, −6 eg ∂w0 = (2/4)(−2 − 3 − 5 − 6) = 0.5 × −16 = −8 eg ∂w1 = 0.5 × (1×−2 + 2×−3 + 3×−5 + 4×−6) = 0.5 × (−2 − 6 − 15 − 24) = 0.5 × −47 = −23.5 eg w0 = 0 − 0.1 × −8 = 0.8; w1 = 0 − 0.1 × −23.5 = 2.35

Sanity: line passes (x̄, ȳ): 0.5 + 1.4 × 2.5 = 4

Trap: compute both ∂ with the OLD w, then update both

Logistic regression (labels ±1)

0) Given w = (0.8, −0.5), b = −1, x = (3, 2), true label +1

1) z = w1x1 + w2x2 + b. eg 0.8 × 3 + (−0.5) × 2 + (−1) = 2.4 − 1 − 1 = 0.4

2) p(+1) = σ(z) = 1/(1 + e^(−z)) eg e^(−0.4) = 0.6703; 1/(1 + 0.6703) = 1/1.6703 = 0.5987 Keys: [SHIFT ln] (−)0.4 [=], then 1 ÷ (1 + [Ans]) [=]

3) Class: z > 0 (p > 0.5) gives +1, else −1. eg 0.4 > 0: +1

4) odds = p/(1 − p) = e^z; log-odds = z. eg e^0.4 = 1.4918

5) Loss = −ln p(true class). eg −ln 0.5987 = 0.5130 eg if true −1: −ln(1 − 0.5987) = −ln 0.4013 = 0.9130

Boundary: w·x + b = 0 (p = 0.5); 1 feature: x* = −b/w

Trap: minus sign in e^(−z); p(−1) = 1 − p(+1)

AdaBoost: one round (labels ±1)

0) Given 10 points; learner 1 gets 3 wrong

1) Start weight w = 1/n. eg 1/10 = 0.1 each

2) ε = ADD the weights of the WRONG points. eg 0.1 + 0.1 + 0.1 = 0.3

3) α = ½ ln((1 − ε)/ε) eg 0.7/0.3 = 2.3333; ln 2.3333 = 0.847298; ÷ 2 = 0.4236 Keys: 0.5 × [ln] 0.7 ÷ 0.3 [)] [=]

4) Reweight: wrong w × e^α, right w × e^(−α) eg e^0.4236 = 1.5275, e^(−0.4236) = 0.6547 ([SHIFT ln] [Ans] [=]) eg wrong 0.1 × 1.5275 = 0.1528; right 0.1 × 0.6547 = 0.0655

5) Renormalise: Z = sum of ALL new weights; each w ÷ Z eg Z = 3 × 0.1528 + 7 × 0.0655 = 0.4584 + 0.4585 = 0.9169 (unrounded 0.9165) eg wrong 0.1528/0.9169 = 0.1666, right 0.0655/0.9169 = 0.0714 (exact 1/6, 1/14)

6) Final: H(x) = sign(α1h1(x) + α2h2(x) + ...), h = each learner's ±1 vote Table: learner | α | h(x) | α × h(x); add last column eg 0.4236 × (+1) + 0.2 × (−1) + 0.3 × (−1) = 0.4236 − 0.2 − 0.3 = −0.0764 < 0: predict −1

Sanity: wrong points now hold half: 3 × 1/6 = 0.5

Trap: ε is a SUM of weights, not count/n (equal only in round 1)

Neural net forward and one backprop gradient

0) Given x = 1; 2 sigmoid hidden units, both w1 = 0; w2 = (1, 1); y = 2; η = 0.1

1) z_j = σ(w1 × x). eg σ(0 × 1) = 1/(1 + e^0) = 1/2 = 0.5 each

2) ŷ = Σ w2_j z_j; J = (y − ŷ)². eg 1 × 0.5 + 1 × 0.5 = 1; (2 − 1)² = 1

3) δout = 2(ŷ − y); ∂J/∂w2_j = δout × z_j. eg 2(1 − 2) = −2; −2 × 0.5 = −1

4) δh = δout × w2_j × z(1 − z); ∂J/∂w1 = δh × x eg −2 × 1 × 0.5 × (1 − 0.5) = −0.5; −0.5 × 1 = −0.5

5) new w = w − η × grad. eg w2 = 1 − 0.1 × (−1) = 1.1; w1 = 0 − 0.1 × (−0.5) = 0.05

Trap: use the OLD w2 in δh

Attention weights

0) Given q, keys k_i, values v_i, d_k. eg q = (1,0,2,1); k1 = (1,1,0,0), k2 = (0,1,1,1), k3 = (2,0,1,0); v = 10, 2, 6; d_k = 4

1) Dot product q·k_i: multiply matching positions, add eg q·k1 = 1+0+0+0 = 1; q·k2 = 0+0+2+1 = 3; q·k3 = 2+0+2+0 = 4

2) Scale: score_i = q·k_i ÷ √d_k. eg √4 = 2: 1/2, 3/2, 4/2 = 0.5, 1.5, 2.0

3) e^ each score [SHIFT ln], add them eg e^0.5 = 1.6487, e^1.5 = 4.4817, e^2 = 7.3891; sum = 13.5195

4) Weight α_i = e^score_i ÷ sum eg 1.6487/13.5195 = 0.1220, 4.4817/13.5195 = 0.3315, 7.3891/13.5195 = 0.5465

5) Output = Σ α_i × v_i eg 0.1220×10 + 0.3315×2 + 0.5465×6 = 1.2195 + 0.6630 + 3.2793 = 5.1618

Sanity: weights sum to 1; output lies between smallest and largest v

How to answer

Explain: claim, reason, example, so-what. Compare: pro/con each, when to use

Justify: deciding property, what fails otherwise. ~1.7 min/mark

Explainers

Why do we need a validation set? To choose models and hyperparameters (α, depth, epochs) and detect overfitting on unseen data. Training error is optimistic, so cannot pick models.

Split validation from the train or the test set? From the TRAINING set. Test is used once at the end; tuning on it leaks test info (data leakage), so the reported score is too optimistic.

Validation vs cross-validation? Hold-out: one split, cheap, high variance, val data never trained on. k-fold CV: train on k-1 folds, validate on 1, rotate, average; all data validated once, lower variance, k× cost.

When stratified or time-series CV? Stratified keeps class ratios per fold (imbalanced data). Time data: validate on a later period, no shuffling.

Why are d+1 params enough for binary logistic? Softmax K=2, divide by e^(w1·x+β1): p(1|x) = 1/(1+e^(-(w1-w2)·x-(β1-β2))). Only differences matter: w = w1-w2, β = β1-β2, so d+1; p(2|x) = 1-p(1|x).

Why is K(d+1) redundant? Adding the same (c, b) to all classes leaves softmax unchanged, so fix one class at 0: (K-1)(d+1) free params.

Why does a bootstrap sample hold ~63.2% of points? Missed in one draw w.p. 1-1/n, in n draws (1-1/n)^n → 1/e ≈ 0.368, so P(chosen) = 1-(1-1/n)^n ≈ 0.632.

What is the out-of-bag (OOB) set for? The ~36.8% not drawn are unseen by that model, so they give a free validation (OOB) error estimate.

ROC points below the diagonal: problem and fix? TPR < FPR, worse than random (AUC < 0.5): scores are inverted. Invert the classifier (use 1-score): (FPR, TPR) becomes (1-FPR, 1-TPR), AUC becomes 1-AUC.

Has k-means converged? Only if an assignment step moves no point, so centres stop moving. If any point switched or a centre moved: No, iterate again.

Why do total conv params rise when the big kernel moves to layer 2? Params = f(C_in k^2 + 1). Layer 1 has C_in = 3, layer 2 has C_in = f1 > 10, so big kernels cost more there: change = f1($s1^2$-$s2^2$)(f2-3) > 0.

Fitting, bias-variance, regularisation

Overfitting vs underfitting? Overfit: train error low, val high, fits noise (high variance). Underfit: both high, too simple (high bias).

How to reduce overfitting? More data, simpler model, L1/L2, pruning, dropout, early stopping, bagging.

L1 vs L2? Penalty α‖w‖ shrinks weights: variance down, bias up a little. L2 $\alpha\Sigma w^2$ shrinks all smoothly; L1 $\alpha\Sigma|w|$ sets some w to 0 = feature selection.

Bias-variance trade-off? Error = $bias^2$ + variance + noise. More complexity lowers bias, raises variance; choose the complexity at minimum validation error.

Metrics

Why is accuracy poor on imbalanced data? All-majority scores 99% yet finds no positives. Use precision, recall, F1, PR-AUC.

Precision vs recall, when each? Precision when false alarms are costly (spam). Recall when misses are costly (cancer, fraud).

Why is F1 a harmonic mean? Pulled to the smaller: P = 1, R = 0.01 gives F1 0.02. Both must be good.

ROC vs PR curve? FPR divides by the many negatives, so ROC looks optimistic when positives are rare. PR curve ignores TN, so it is more informative on imbalanced data.

What do ROC and AUC show? ROC = TPR vs FPR over all thresholds; diagonal random, top-left perfect. AUC = P(random positive scored above random negative); 0.5 random, 1 perfect.

Bayes and naive Bayes

Naive Bayes assumption? Features independent given the class: p(x|C) = Π p(xj|C); d easy 1-D estimates; Gaussian NB Σ is diagonal.

Why Laplace smoothing? An unseen word gives P = 0 and zeroes the product; (count + 1)/(total + |V|) keeps all > 0.

Generative vs discriminative? Generative models p(x|C)P(C) (NB, GAN); discriminative models p(C|x) directly (logistic, trees, NN).

MLE vs MAP? MLE = argmax P(D|h), data only. MAP = argmax P(D|h)P(h), adds a prior. Uniform prior: MAP = MLE. MAP safer with little data.

Decision trees

What do entropy and IG measure? H = label impurity, 0 pure, 1 bit at 50/50. IG = entropy drop from a split; ID3 splits on max IG.

Why does IG favour many-valued attributes? An ID splits into tiny pure subsets: max IG, useless. Gain ratio = IG/SplitInfo.

Why do trees overfit and how to stop it? They can split until every training row is right, fitting noise (high variance). Pre-prune (max depth, min leaf size), post-prune on validation, or use RF.

Ensembles

Bagging vs boosting? Bagging: parallel, bootstrap samples, equal vote, cuts variance (RF). Boosting: sequential, focuses on errors, α-weighted vote, cuts bias; noise-sensitive.

Why random features in a random forest? Each split tries ≈√d random features, so trees decorrelate; averaging cuts more variance.

Why do ensembles work? Combining diverse models cancels independent errors. Base models must beat chance (each 45% accurate: more voters is worse).

Neural networks

Why a non-linear activation? Else the layers collapse into one linear map: no XOR.

Vanishing gradient and fixes? Backprop multiplies σ' ≤ 0.25 many times, so early layers stop learning. ReLU, batch norm, skip connections, LSTM.

How does dropout reduce overfitting? Drops random units in training so none rely on others; like averaging many sub-nets. Test: all units, × keep prob α.

Batch vs SGD vs mini-batch? Batch: all data per update, stable, slow. SGD: 1 sample, fast, noisy zig-zag. Mini-batch: balance, the default.

CNN

Why CNN over FNN for images? Local connectivity + weight sharing: far fewer params, translation invariance, feature hierarchy.

Pooling, padding, stride? Pooling downsamples, robust to shifts, no weights. Padding keeps size and borders; bigger stride shrinks output.

Why ResNet skip connections? Block outputs F(x) + x: learns only the residual; gradient flows straight through the + to early layers, so 100+ layers train.

RNN, attention, transformer

RNN vs LSTM? RNN gradient ~ W^T vanishes or explodes over long sequences; LSTM gates + additive cell state keep it.

Why self-attention over RNN? Every token links directly to every other, in parallel, no vanishing through time; cost $O(n^2)$.

Why scale by √dk? q·k grows with dk; scaling stops softmax saturating (tiny gradients).

BERT vs GPT? BERT: encoder, bidirectional, masked words: understanding. GPT: decoder, causal, next token: generation.

Why multi-head? Each head has its own Q, K, V projections, so each learns a different relation (syntax, coreference); concat × W^O.

Why positional encoding? Attention is order-blind; added sin/cos position vectors give word order.

Why mask the decoder? Token t must not see the future it predicts: scores j > t set to -∞.

Generative AI

GAN, VAE, diffusion? GAN: generator vs discriminator, min-max. VAE: encoder gives μ, σ, z = μ + σε, decoder rebuilds. Diffusion: learn to remove noise step by step.

Why RLHF after fine-tuning? SFT copies demos, cannot rank answers; human rankings train a reward model, PPO maximises reward minus KL.

Unsupervised

k-means weaknesses? Must pick k; init-sensitive (restarts, k-means++); assumes round equal clusters; outliers; needs scaling.

How to choose k? Elbow (bend in SSE vs k) or highest average silhouette (b − a)/max(a, b).

What does PCA do? Projects centred data onto the top-k max-variance directions (covariance eigenvectors); var explained = Σtop λ / Σλ.

k-means vs hierarchical? k-means: k in advance, fast, random init. Agglomerative: merge closest pair, dendrogram cut at any k, deterministic, $O(n^2)$.